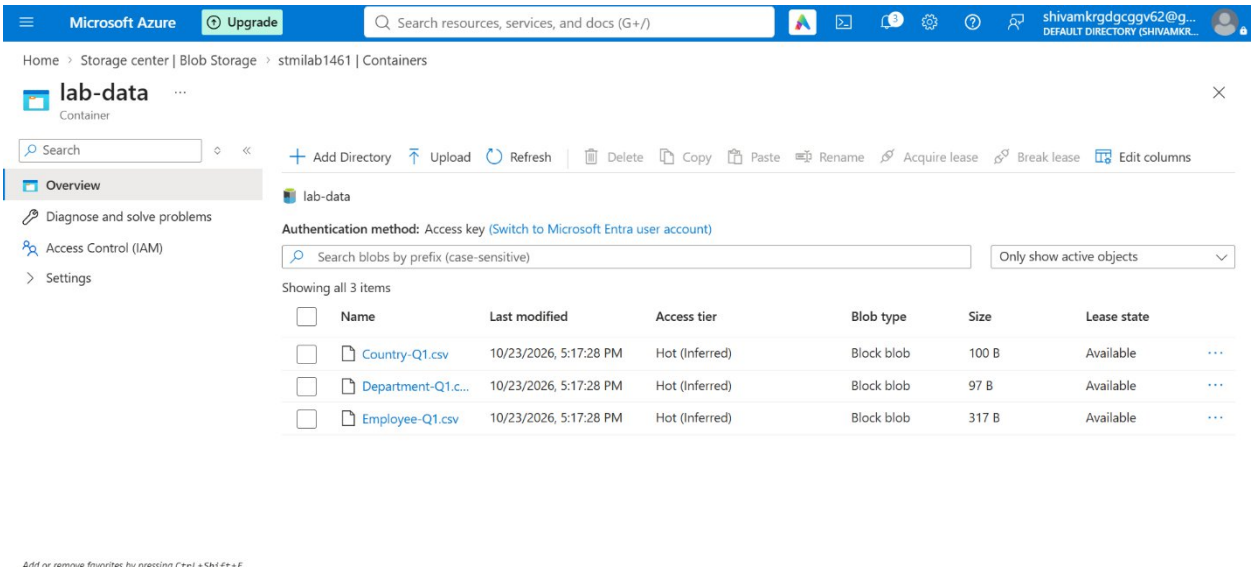


SESSION 4 — AZURE II: SECURITY AND DATA INTEGRATION

- **Topic:** Azure II — Security and Data Integration
- **Date:** 26th October 2026

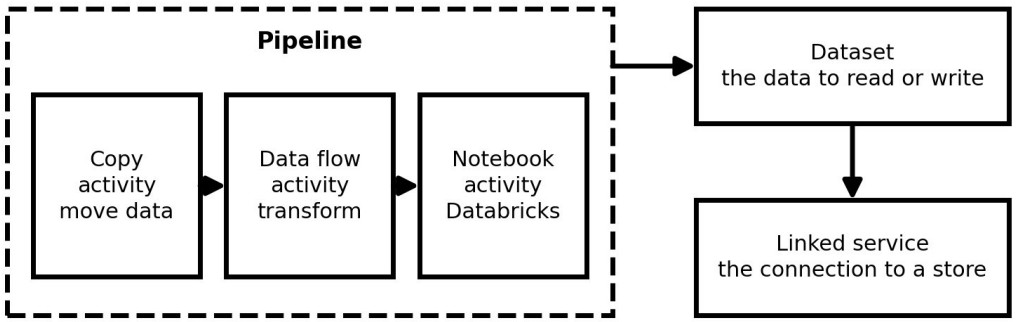
12. Azure Data Factory: Pipelines, Linked Services, and Activities

Azure Data Factory (ADF) is Azure's managed service for moving and preparing data. You describe the work once, for example "copy yesterday's files from this storage account, clean them, and load them into that one", and Data Factory runs it for you, on a schedule or when something happens, without any server for you to look after. This kind of work is called data integration, and the copy, clean, and load pattern is often written as ETL.

Building block	Simple meaning
Pipeline	A group of activities that together do one job, such as "load daily sales". It is the thing you run, schedule, and monitor.
Activity	One step inside a pipeline, such as copying files or running a notebook. Activities can run one after another or side by side.
Linked service	A saved connection to a store or a compute service, like a connection string. It holds where to connect and how to sign in, for example to a storage account, a database, Key Vault, or Azure Databricks.
Dataset	A named pointer to the data an activity reads or writes, such as one folder or file format inside a linked storage account. It answers which data, while the linked service answers where.
Integration runtime (IR)	The compute that actually runs the work. The Azure IR is the default and

	is managed for you. A self-hosted IR is installed on your own machine so Data Factory can reach data inside a private or on-premises network.
Parameter	A value given to a pipeline when it starts, such as a date or a folder name, so that the same pipeline can be reused for different inputs.

Azure Data Factory: how the parts fit together



Activities in a pipeline use datasets, and datasets reach data through linked services

Diagram 9: A pipeline holds activities, activities use datasets, and datasets reach the data through linked services.

Activity types. Activities fall into three groups. Data movement activities copy data, transformation activities change it, and control activities decide the order and the logic of the run.

Activity	**What it does**
Copy (data movement)	Reads from a source and writes to a sink, for example from a database into the bronze container of ADLS Gen2. It is the most used activity.
Data flow (transformation)	Cleans and reshapes data with steps drawn on a canvas. Data Factory runs them on managed Spark, so no code is written.
Notebook (transformation)	Runs an Azure Databricks notebook, so that the heavy transformation is done in Databricks and Data Factory only starts it and waits.
Lookup and Get Metadata	Read a value or details about a file or folder (name, size, last modified) so that later steps can use them.
ForEach and If Condition	Control activities. ForEach repeats steps for every item in a list, and If Condition chooses between two paths.
Execute Pipeline	Starts another pipeline from inside this one, so large jobs can be split into small reusable pipelines.
Web and Wait	Web calls a URL or an API, and Wait pauses the pipeline for a set time.

Order and failure. Each arrow between two activities carries a condition: run the next step on success, on failure, when skipped, or on completion whichever way it ended. A failure path is how a pipeline sends an alert or cleans up. An activity can also be given a retry count and a timeout, so a short network problem does not fail the whole run.

Step	What to do
1. Create the factory	In the Azure portal create a Data Factory in the resource group, then open Azure Data Factory Studio. Its three main places are Author (build), Monitor (watch runs), and Manage (connections and settings).
2. Create linked services	Add one for the source and one for the destination, such as the ADLS Gen2 account stdatalakedev001. Choose the managed identity sign-in and not an account key.
3. Create datasets	Point a dataset at each folder or file, for example bronze/sales as CSV, and bronze/sales_clean as Parquet.
4. Build the pipeline	Drag a Copy activity onto the canvas, choose the source and sink datasets, and add a Notebook activity after it if a transformation is needed.
5. Debug	Use Debug to run it from the authoring page and read the result for each activity before anything is scheduled.
6. Publish and run	Publish saves the work to the service. Only published pipelines can be started by a trigger (section 13). Check every run under Monitor.

- **Secrets belong in Key Vault.** When a source needs a password or a connection string, store it as a Key Vault secret and make the linked service read it from there . The Data Factory identity then needs the Key Vault Secrets User role on the vault , and the secret never appears inside the pipeline.

13. Azure Data Factory Triggers and Pipeline Scheduling

A pipeline does nothing by itself. A trigger is the rule that says when a pipeline starts, such as every night at 2 a.m., or when a new file lands in storage. Each time a trigger fires, it starts one pipeline run, and a trigger can pass values such as the date or file name into the pipeline's parameters. You can also start a pipeline by hand with Trigger now, which is called a manual or on-demand run.

Trigger type	Simple meaning
Schedule trigger	Starts pipelines on a wall-clock schedule, such as every day at 02:00 or every Monday to Friday at 06:00. It looks forward only: it fires from now on and does not run for earlier periods.
Tumbling window trigger	Cuts time into fixed, separate slices of equal size, such as one hour each, and runs the pipeline once for every slice. It remembers each slice, so a missed or failed slice can be run again.

Storage events trigger	Starts a pipeline when a file is created or deleted in a Blob or ADLS Gen2 storage account. It can be limited to a path, for example files that start with bronze/sales/.
Custom events trigger	Starts a pipeline when an application sends its own event to Azure Event Grid. It is used when the starting signal is not a file.

Triggers decide when a pipeline starts

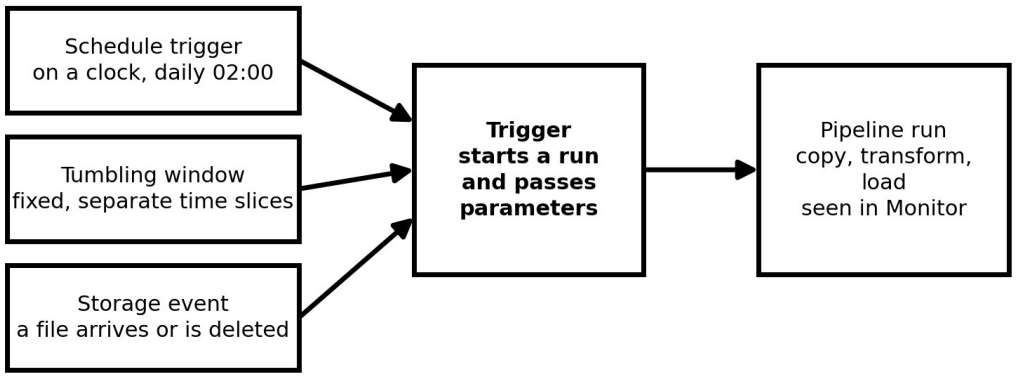


Diagram 10: Different kinds of triggers all do the same thing, which is to start a pipeline run.

Choosing between them. The schedule and tumbling window triggers both run on time, and the difference is how they treat the time.

Point	**Schedule trigger vs. tumbling window trigger**
Past periods	Schedule: runs only from now on, so an earlier period is not processed unless you start it by hand. Tumbling window: can start from a past date and work through every earlier slice, which is called backfill.
Failures and reruns	Schedule: each firing is independent, and a failed run is rerun by hand from Monitor. Tumbling window: each slice has its own status and a retry policy, so a failed slice can be retried or rerun alone.
Which time the run is for	Schedule: the pipeline gets the time it was scheduled to fire, available as @trigger().scheduledTime. Tumbling window: the pipeline gets the start and end of its slice, available as @trigger().outputs.windowStartTime and windowEndTime.
Link to pipelines	Schedule: one trigger can start several pipelines, and one pipeline can have several triggers. Tumbling window: one trigger belongs to exactly one pipeline, but triggers can wait for each other and a limit can be set on how many slices run at once.

Making a trigger work. Three things must be true before a trigger fires. The pipeline must be published, the trigger must be published, and the trigger must be started, because a new trigger is stopped by default. A schedule trigger that is stopped does not catch up on the runs it missed when it is started again. The trigger's values are connected to the pipeline's parameters, for example a parameter run_date given the value @trigger().scheduledTime, so the same pipeline works for any day. Every firing appears under Monitor, in Trigger runs, and links to the pipeline run it started.

- **Choose the time zone on purpose.** A schedule trigger has a start date, an end date, a time zone, and a repeat of every number of minutes, hours, days, weeks, or months. State the time zone clearly, because "2 a.m." means a different moment in different places.
- **Use events for arriving files.** If data arrives at unpredictable times, a storage events trigger is better than a nightly schedule that may run before the file is there. The file path and name are passed in as @triggerBody().folderPath and @triggerBody().fileName.
- **Use tumbling windows for history.** When each run must cover exactly one hour or one day of data, and none may be skipped or doubled, use a tumbling window trigger.